IE0005 Mini Project

Alzheimer's Disease Prediction

through AI & Data Science

Jia Wei
Hudson
Anu
Randall
Group Brown EL12

Dataset Chosen

🧠 Alzheimer's Disease Dataset 🧠" from Kaggle

- by Rabie El Kharoua
- Published In 2024
- Source: https://www.kaggle.com/datasets/rabieelkharoua/alzheimers-disease-dataset

Why?

There is **rapid growth** in Alzheimer's cases across the world with the projected number of patients to increase from 41.27 Mil in 2020 to 106.23 Mil in 2050. **Early detection (predicting) and treatment** can **significantly improve the quality of life**.

Medical data is often complex with mixture of multiple variables of different class (multi class and multi variate), ML models would be suitable in predicting if a patient has Alzheimer's based on their data.

Problem: How can we best predict if an individual has Alzheimer’s Disease based on patient data? Which ML Prediction Model is the most suitable?
(Binary Classification Problem)

01

Data Preparation

Data Preparation (I)

Cleaning of Data

I. Remove unnecessary Columns (Doctor in Charge - values are NaN)

II. Check for missing values or duplicated values (No missing/ duplicated values found)

III. Convert Data Types as appropriate according to column header description (e.g gender 0-1 converted to category)

IV. Rebase index to start from 1 for ease of reference

```
Data columns (total 35 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   PatientID                  2149 non-null   int64
 1   Age                        2149 non-null   int64
 2   Gender                     2149 non-null   int64
 3   Ethnicity                  2149 non-null   int64
 4   EducationLevel             2149 non-null   int64
 5   BMI                        2149 non-null   float64
 6   Smoking                    2149 non-null   int64
 7   AlcoholConsumption         2149 non-null   float64
 8   PhysicalActivity           2149 non-null   float64
 9   DietQuality                2149 non-null   float64
 10  SleepQuality               2149 non-null   float64
 11  FamilyHistoryAlzheimers    2149 non-null   int64
 12  CardiovascularDisease      2149 non-null   int64
 13  Diabetes                   2149 non-null   int64
 14  Depression                 2149 non-null   int64
 15  HeadInjury                 2149 non-null   int64
 16  Hypertension               2149 non-null   int64
 17  SystolicBP                 2149 non-null   int64
 18  DiastolicBP                2149 non-null   int64
 19  CholesterolTotal           2149 non-null   float64
 20  CholesterolLDL             2149 non-null   float64
 21  CholesterolHDL             2149 non-null   float64
 22  CholesterolTriglycerides   2149 non-null   float64
 23  MMSE                       2149 non-null   float64
 24  FunctionalAssessment       2149 non-null   float64
 25  MemoryComplaints           2149 non-null   int64
 26  BehavioralProblems         2149 non-null   int64
 27  ADL                        2149 non-null   float64
 28  Confusion                  2149 non-null   int64
 29  Disorientation             2149 non-null   int64
 30  PersonalityChanges         2149 non-null   int64
 31  DifficultyCompletingTasks  2149 non-null   int64
 32  Forgetfulness              2149 non-null   int64
 33  Diagnosis                  2149 non-null   int64
 34  DoctorInCharge             2149 non-null   object
dtypes: float64(12), int64(22), object(1)
memory usage: 587.7+ KB
```

Original Dataset

```
New dtypes:

PatientID                    int64
Age                          int64
Gender                    category
Ethnicity                 category
EducationLevel            category
BMI                        float64
Smoking                       bool
AlcoholConsumption         float64
PhysicalActivity           float64
DietQuality                float64
SleepQuality               float64
FamilyHistoryAlzheimers       bool
CardiovascularDisease         bool
Diabetes                      bool
Depression                    bool
HeadInjury                    bool
Hypertension                  bool
SystolicBP                   int64
DiastolicBP                  int64
CholesterolTotal           float64
CholesterolLDL             float64
CholesterolHDL             float64
CholesterolTriglycerides   float64
MMSE                       float64
FunctionalAssessment       float64
MemoryComplaints              bool
BehavioralProblems            bool
ADL                        float64
Confusion                     bool
Disorientation                bool
PersonalityChanges            bool
DifficultyCompletingTasks     bool
Forgetfulness                 bool
Diagnosis                     bool
dtype: object
```

After cleaning

Data Preparation (II)

Separating Data into new DF for exploratory analysis based on dtype

```
Boolean DataFrame columns:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2149 entries, 0 to 2148
Data columns (total 15 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Smoking                    2149 non-null   bool
 1   FamilyHistoryAlzheimers    2149 non-null   bool
 2   CardiovascularDisease      2149 non-null   bool
 3   Diabetes                   2149 non-null   bool
 4   Depression                 2149 non-null   bool
 5   HeadInjury                 2149 non-null   bool
 6   Hypertension               2149 non-null   bool
 7   MemoryComplaints           2149 non-null   bool
 8   BehavioralProblems         2149 non-null   bool
 9   Confusion                  2149 non-null   bool
 10  Disorientation             2149 non-null   bool
 11  PersonalityChanges         2149 non-null   bool
 12  DifficultyCompletingTasks  2149 non-null   bool
 13  Forgetfulness              2149 non-null   bool
 14  Diagnosis                  2149 non-null   bool
dtypes: bool(15)
memory usage: 31.6 KB
None
```

Boolean DF

```
Categorical DataFrame columns:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2149 entries, 0 to 2148
Data columns (total 3 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Gender          2149 non-null   category
 1   Ethnicity       2149 non-null   category
 2   EducationLevel  2149 non-null   category
dtypes: category(3)
memory usage: 6.9 KB
None
```

Categorical DF

```
Numerical DataFrame columns:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2149 entries, 0 to 2148
Data columns (total 15 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Age                        2149 non-null   int64
 1   BMI                        2149 non-null   float64
 2   AlcoholConsumption         2149 non-null   float64
 3   PhysicalActivity           2149 non-null   float64
 4   DietQuality                2149 non-null   float64
 5   SleepQuality               2149 non-null   float64
 6   SystolicBP                 2149 non-null   int64
 7   DiastolicBP                2149 non-null   int64
 8   CholesterolTotal           2149 non-null   float64
 9   CholesterolLDL             2149 non-null   float64
 10  CholesterolHDL             2149 non-null   float64
 11  CholesterolTriglycerides   2149 non-null   float64
 12  MMSE                       2149 non-null   float64
 13  FunctionalAssessment       2149 non-null   float64
 14  ADL                        2149 non-null   float64
dtypes: float64(12), int64(3)
memory usage: 252.0 KB
None
```

Numeric DF

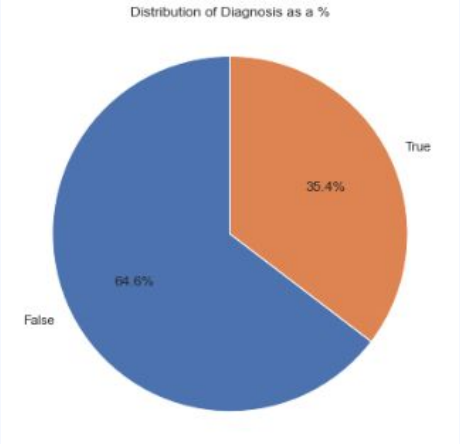
02

Data Analysis

Univariate - Bivariate - Multivariate

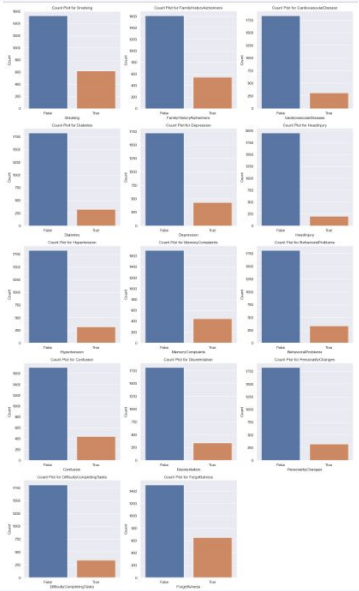
Boolean (Uni-variate)

Diagnosis (Response variable)



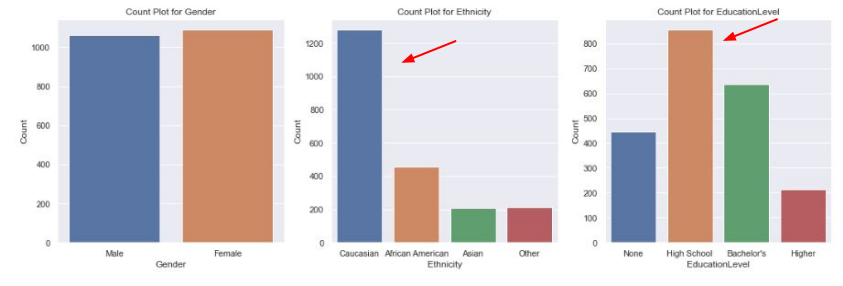
Dataset moderately imbalanced, with about majority (2/3) diagnosis - False
False >>> True

Other Boolean predictor variables



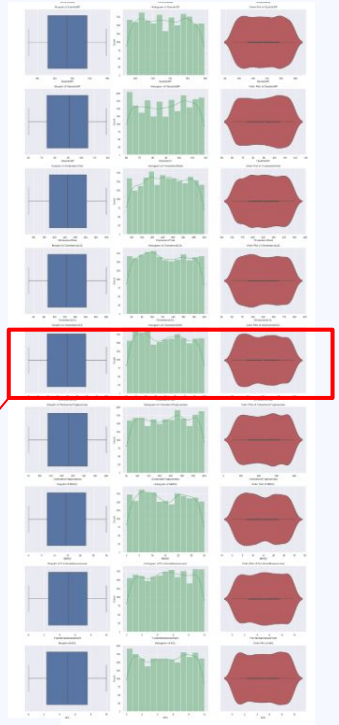
Also imbalanced, individuals in dataset generally healthy, False >>> True

Categorical & Numerical (Uni-variate)



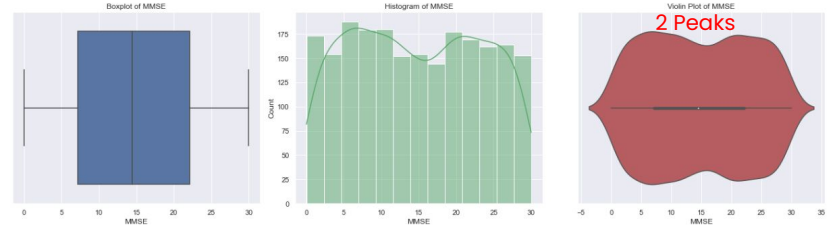
Categorical

1. Gender - equal distribution
2. Ethnicity - Caucasian dominant
3. Education level - High school majority



Numerical

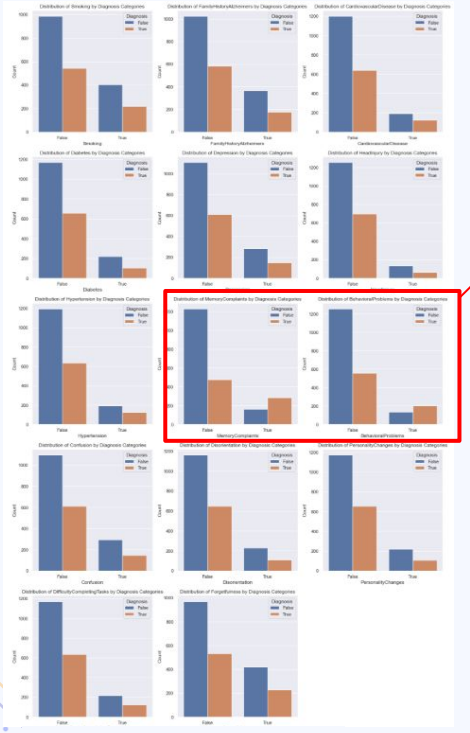
- Mostly uniform distribution
- Dips at either extremes (min/max)



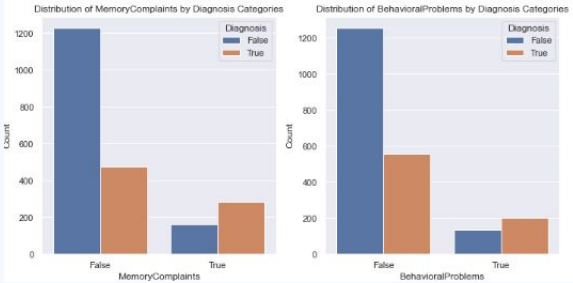
MMSE (Numerical)

Bi modal distribution - 2 groups of data

Boolean & Categorical (Bi-variate)

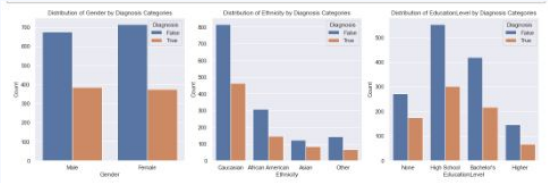


Boolean



MemoryComplaints & BehaviouralProblems

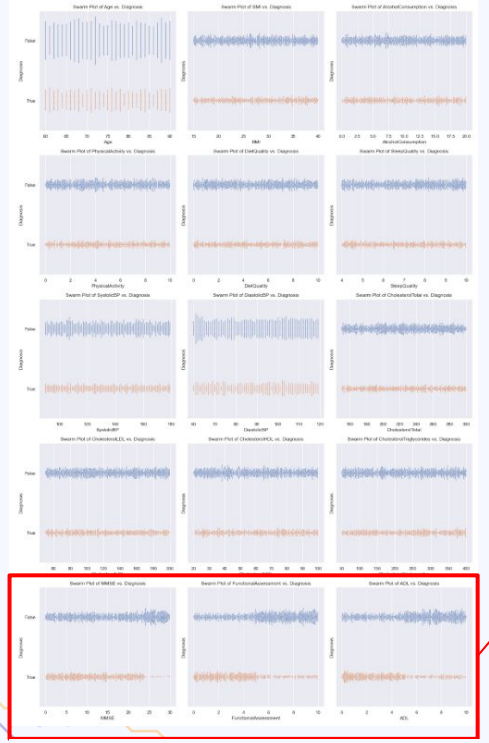
For both variables - True, there is greater number of **Diagnosis - True than Diagnosis - False**



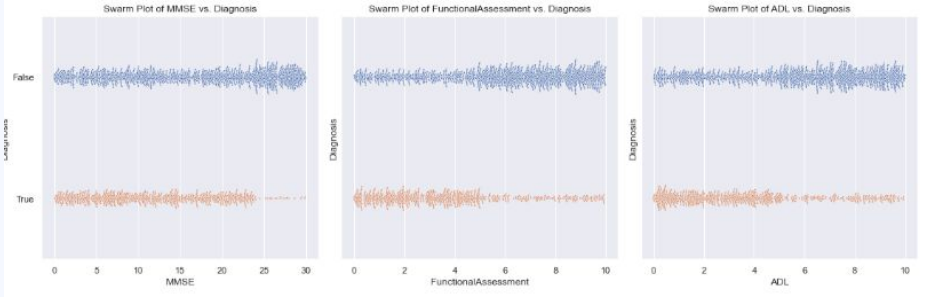
Categorical

- For each **boolean value** (True/False) and **each item under a category** (e.g male female under gender), the **ratio/distribution of Diagnosis - False to Diagnosis - True** is generally the same, with greater number of False Diagnosis than True

Numerical(Bi-variate)



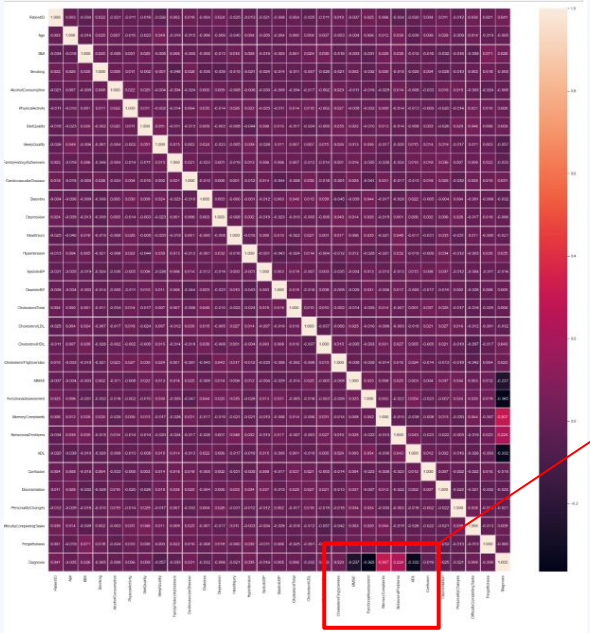
Numerical (Swarm plot)



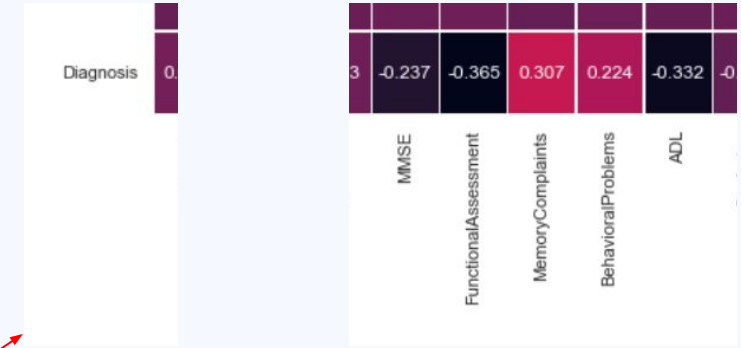
- For **MMSE,** and **Functional Assessment or ADL**, a distinct **"cut"** or **separation** between Diagnosis - True and Diagnosis - False Swarm plots is observed at ~24 and ~5 respectively.

- Useful in a **decision tree** in predicting whether an individual is likely to be diagnosed with Alzheimer's Disease.

Correlation matrix (Multi-variate)



Correlation Matrix



- High positive correlation to Diagnosis for BehavioralProblems (0.224) and MemoryComplaints (0.307) and High negative correlation for FunctionalAssessment (-0.365), ADL (-0.332) and MMSE (-0.237) could be **useful for predicting Diagnosis** (of Alzheimer's Disease)

- The correlation of the remaining variable **among themselves** is low. Might not be useful in **directly predicting** diagnosis

- The **interactions** between the different variables **taken together** could yield more accurate prediction models and **should not be discarded**.

Data Pre-Processing

Normalization of Data

```
#normalize the columns such that the
columns = ['Age', 'BMI', 'AlcoholConsumption', 'PhysicalActivity', 'DietQuality', 'SleepQuality', 'SystolicBP', 'DiastolicBP', '

min_max_scaler = MinMaxScaler()
Alzheimers_DF[columns] = min_max_scaler.fit_transform(Alzheimers_DF[columns])

Alzheimers_DF
```

Split into train test set

```
#split data into features and target
X = Alzheimers_DF.drop(columns = ['Diagnosis'])
y = Alzheimers_DF['Diagnosis']

#split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.30, random_state = 42, shuffle = True)
```

30/70 Ratio to ensure balance and sufficient data for both

03 Classification Models

Machine Learning Models We Tested

Decision Tree
Gradient Boost
Random Forest
Neural Network

Interpreting Metrics

Accuracy

Even though accuracy seems high it can be **misleading** as the **dataset is imbalanced**, and the model could simply **predict the majority** class (Diagnosis - False) and still achieve a high accuracy, without identifying **minority class (Diagnosis - True)**

Goodness of Fit of Model
Accuracy
AUC ROC
AUC PR
F1 Score
F0.5 Score
F2 Score

AUC ROC VS AUC PR

AUC ROC useful in measuring ability to **discriminate between classes** across classification thresholds, more suited for **balanced datasets** where both response classes matter equally

AUC PR focuses more on performance with respect to the **minority class** (Diagnosis - True) i.e **reliability of positive predictions** across thresholds, and suited for **imbalanced data set** and where **misclassifying the minority class** would be very costly

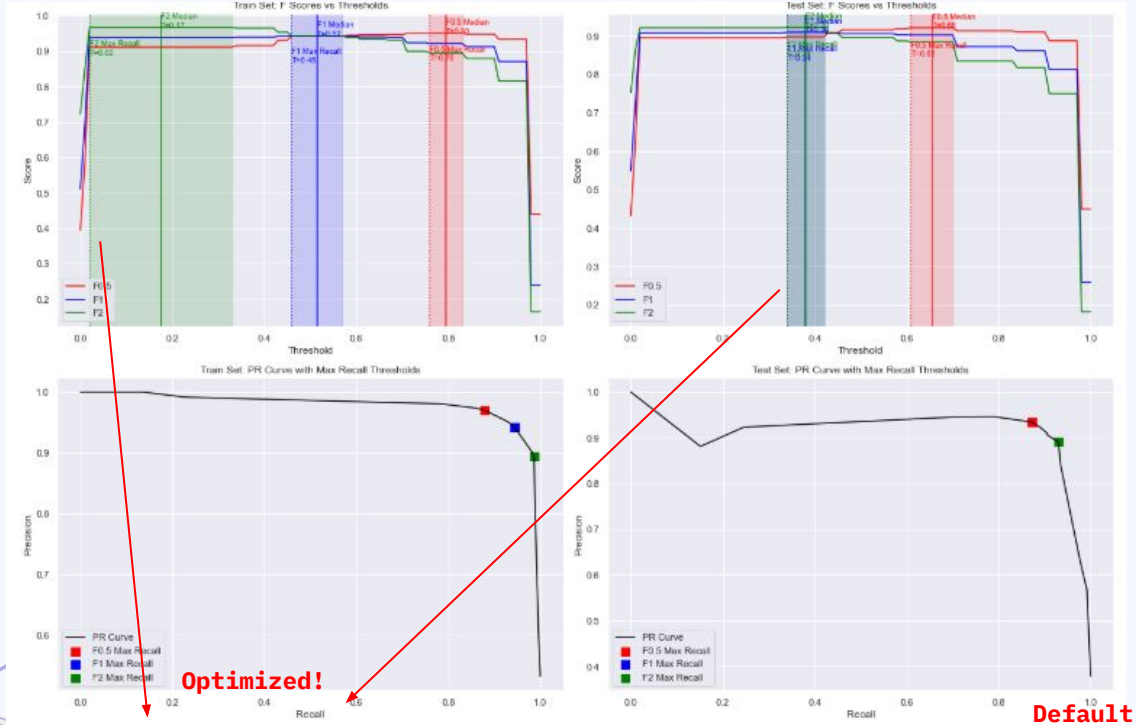
F1 Score
F0.5 Score
F2 Score

F-scores are single metric for **weighted harmonic means of precision and recall** (combine precision and recall) at a threshold value (default 0.5)

Use F2 Score (not F1 and F0.5) as it **prioritizes recall over precision i.e** for **medical context** it is vital to **minimize false negatives (missed diagnoses) even if it means accepting more false positives (over-diagnosis).**

F-Score	Emphasis	Cost Preference	β Relationship
F1	Balanced	Balance FP and FN	β = 1
F0.5	Precision > Recall (Favors Precision)	Avoid false positives	β < 1 (e.g., 0.5)
F2	Recall > Precision (Favors Recall)	Avoid false negatives	β > 1 (e.g., 2)

Maximum F2 Score (Recall optimized)



Optimized!

```
Maximum F2 Scores optimized for recall (Decision Tree)
Train Set: 0.9666 at threshold 0.02
Test Set: 0.9220 at threshold 0.34
```

Key Comparison Metric!

F2 score in previous table is for default **neutral 0.5 threshold**

We can vary the **threshold and find the maximum F2 score**

If there are **multiple thresholds** for which F2 score is the maximum, we take the lowest threshold value as it **biases towards recall**

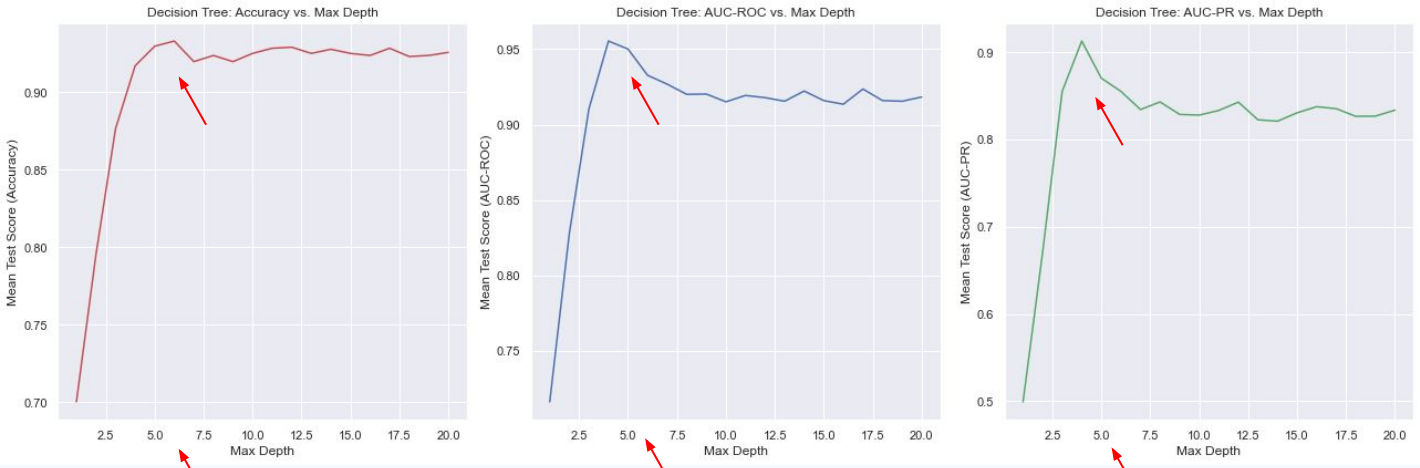
Model Evaluation Metrics (Decision Tree)

Goodness of Fit of Model	Train Dataset	Test Dataset
Accuracy	0.961436170212766	0.928682170542636
AUC ROC	0.989719659165804	0.956594170311925
AUC PR	0.973963671191328	0.904621031241926
F1 Score	0.943798449612403	0.904166666666667
F0.5 Score	0.943798449612403	0.913299663299663
F2 Score	0.943798449612403	0.895214521452145

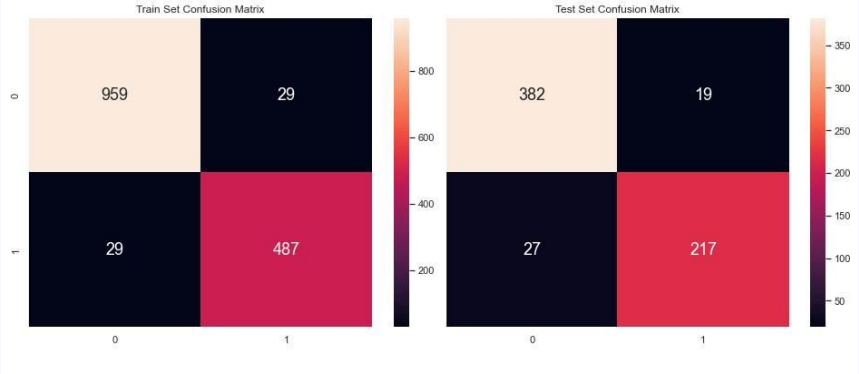
Default 0.5 threshold, not optimized!

Decision Tree

GridsearchCV to exhaustively search for **optimal max depth** of decision tree, optimized based on Accuracy, Area under PR curve, Area under ROC Curve.



Decision Tree



Confusion matrix (Default threshold 0.5)

Model Evaluation Metrics (Decision Tree)		
Goodness of Fit of Model	Train Dataset	Test Dataset
Accuracy	0.961436170212766	0.928682170542636
AUC ROC	0.989719659165804	0.956594170311925
AUC PR	0.973963671191328	0.904621031241926
F1 Score	0.943798449612403	0.904166666666667
F0.5 Score	0.943798449612403	0.913299663299663
F2 Score	0.943798449612403	0.895214521452145

Evaluation Metrics (Default threshold 0.5)

```
Maximum F2 Scores optimized for recall (Decision Tree)
Train Set: 0.9666 at threshold 0.02
Test Set: 0.9220 at threshold 0.34
```

Maximum F2 score optimized for Recall

Gradient Boosting

An ensemble learning method where it builds a model sequentially, correcting the errors of the previous ones.

Advantages

- **Higher Accuracy**: Each tree corrects the mistakes of the previous trees, allowing the prediction to be more accurate
- **Reduces Bias and Variance**

Trade-off

- **More Computationally Expensive**: Training gradient boost models can take longer to train than simple decision trees, especially for larger datasets.

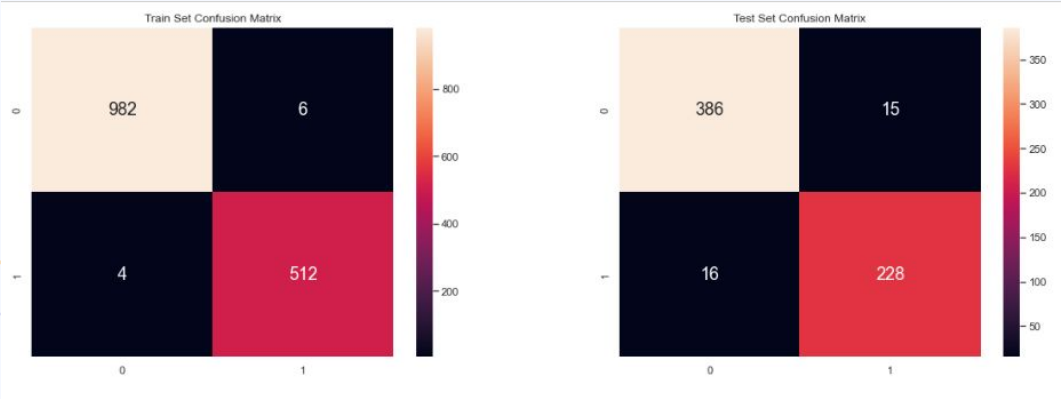
Gradient Boosting

Bayesian Optimisation for hyperparameter tuning

- More efficient than GridSearchCV as it uses a probability model instead of brute forcing a solution
- Optimize for AUC PR

Parameters

- n_estimators: Controls the number of trees
- Learning Rate: Controls how much each tree contributes to learning
- Max_depth: Controls the max depth of each tree
- Subsample: Controls the % of data to use per tree



Model Evaluation Metrics (Gradient Boost)		
Goodness of Fit of Model	Train Dataset	Test Dataset
Accuracy	0.993351063829787	0.951937984496124
AUC ROC	0.999813655336911	0.978343076734393
AUC PR	0.999641239026162	0.955268891933157
F1 Score	0.990328820116054	0.936344969199179
F0.5 Score	0.989180834621329	0.937500000000000
F2 Score	0.991479473276530	0.935192780968007

```
Maximum F2 Scores optimized for recall (Gradient Boost)
Train Set: 0.9969 at threshold 0.45
Test Set: 0.9626 at threshold 0.31
```

Random Forest

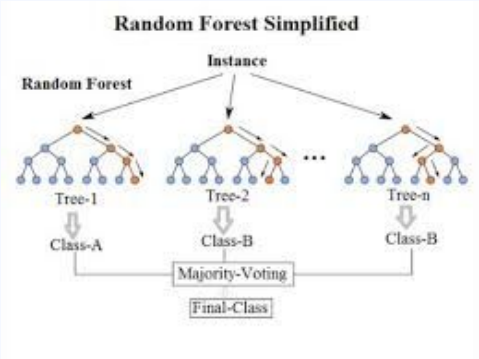
Ensemble learning method that creates multiple decision trees and combines their outputs to improve accuracy and reduce overfitting

Advantages:

- Handles overfitting by averaging multiple decision trees
- Works well with both classification and regression trees

Disadvantages:

- Can be computationally expensive with a large number of trees
- Less interpretable compared to a single decision tree



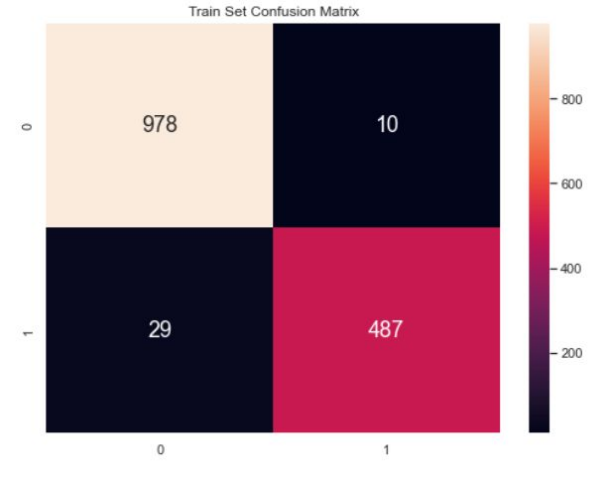
Random Forest

```
# Define the hyperparameter search space
n_estimators = trial.suggest_int('n_estimators', 50, 500)
max_depth = trial.suggest_int('max_depth', 4, 10)
min_samples_split = trial.suggest_int('min_samples_split', 2, 10)
min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 10)
max_features = trial.suggest_categorical('max_features', ['auto', 'sqrt', 'log2'])
```

- n_estimators: More trees -> better performance, higher cost
- max_depth: Limits tree size -> reduces overfitting
- min_samples_split: Controls when a node splits -> avoids overfitting
- min_samples_leaf: Prevents leaf from being too small -> improves generalisation
- max_features: Choose features per split -> adds diversity to trees

- We used the Bayesian Optimisation via the Optuna library, to automatically search for the best set of hyperparameters according to AUC PR. It finds the best settings by learning from previous trials

Random Forest



Model Evaluation Metrics (Random Forest)		
Goodness of Fit of Model	Train Dataset	Test Dataset
Accuracy	0.974069148936170	0.886821705426357
AUC ROC	0.998507281172520	0.977433465516536
AUC PR	0.997010393273809	0.950041725536357
F1 Score	0.961500493583416	0.831408775981524
F0.5 Score	0.972444089456869	0.900000000000000
F2 Score	0.950800468566966	0.772532188841202

```
Maximum F2 Scores optimized for recall (Random Forest)
Train Set: 0.9889 at threshold 0.36
Test Set: 0.9437 at threshold 0.30
```

Neural Network

Type

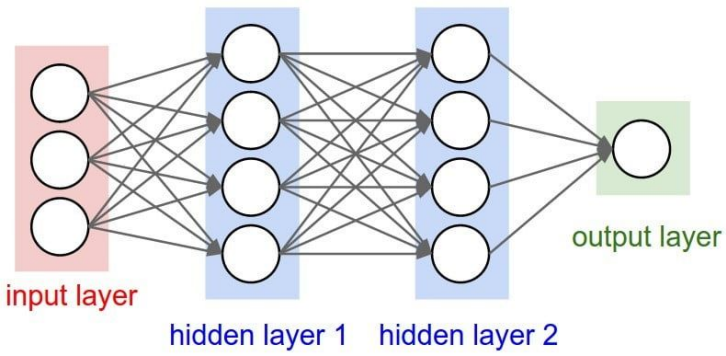
- Fully connected FeedForward Neural Network
- Multilayer Perceptron (MLP)

Feed forward means that the neural network **does not loop back in on itself**.

Multilayer Perceptron (MLP) means each **neuron in one layer is connected to every neuron in the next** i.e fully connected.

Method

- Deep Neural Network



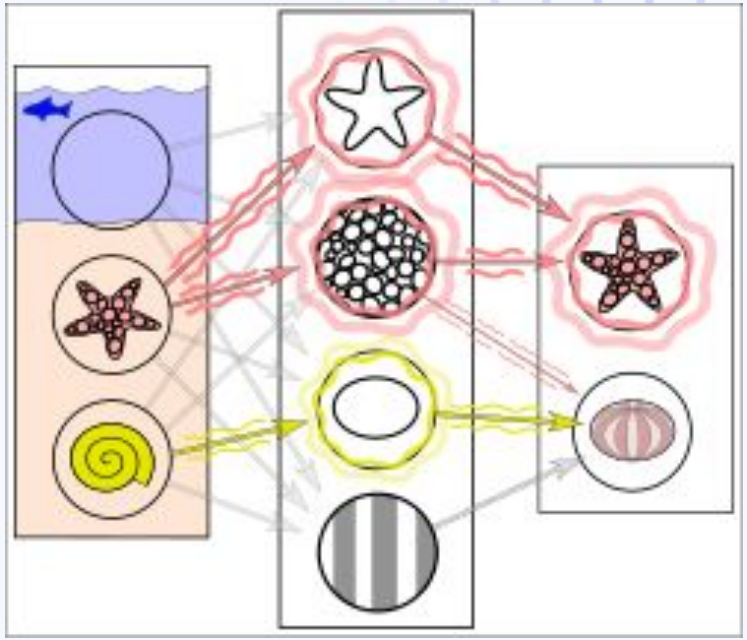
Neural Network

Advantages

- Easy to implement architecture
- Computationally efficient because data only flows one way without loops
- Works well with structured data

Disadvantages

- Cannot handle time-series data (e.g. language models, stock prediction)
- Fixed input size requirement
- Black box nature



Neural Network Hyperparameters

Parameters

- Input layer with 290 neurons and SELU activation (5 total layers)
- Three hidden layers each with 290 neurons and SELU activation
- Each layer followed by Batch Normalization and Dropout (rate = 0.2558)
- Output layer with 1 neuron and Sigmoid activation to output a probability

Compilation

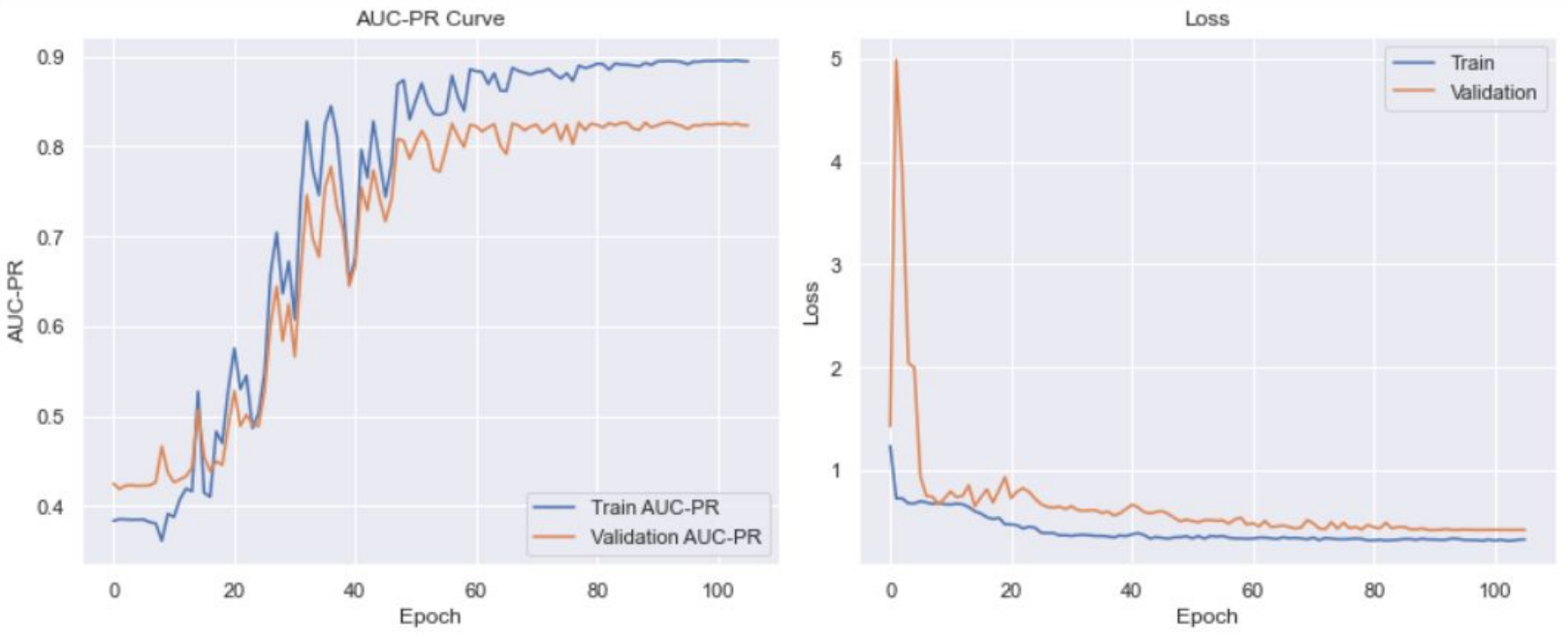
- Adam optimizer with learning rate of 0.0036 and binary cross-entropy loss function (BCE)

Training

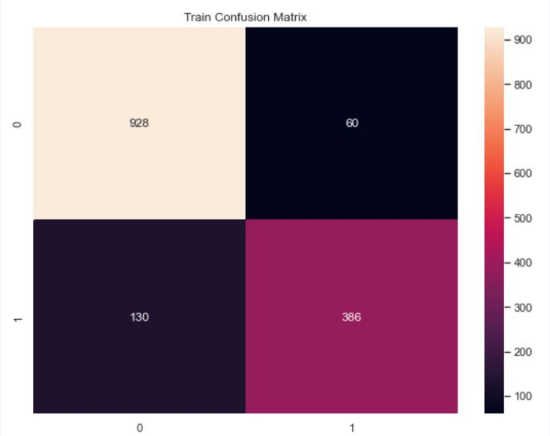
- 200 epochs with batch size of 128

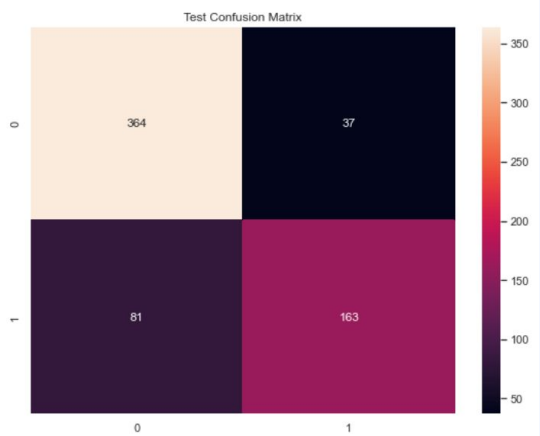
```
Epoch 1/50
24/24 [==============================] - 0s 696us/step - loss: 78.2079 - accuracy: 0.6569
Epoch 2/50
24/24 [==============================] - 0s 696us/step - loss: 16.8220 - accuracy: 0.5991
Epoch 3/50
24/24 [==============================] - 0s 696us/step - loss: 1.4566 - accuracy: 0.5745
Epoch 4/50
24/24 [==============================] - 0s 653us/step - loss: 0.7193 - accuracy: 0.5824
Epoch 5/50
24/24 [==============================] - 0s 653us/step - loss: 0.6764 - accuracy: 0.6084
Epoch 6/50
24/24 [==============================] - 0s 696us/step - loss: 0.6363 - accuracy: 0.6589
Epoch 7/50
24/24 [==============================] - 0s 653us/step - loss: 0.6205 - accuracy: 0.6622
Epoch 8/50
24/24 [==============================] - 0s 653us/step - loss: 0.6233 - accuracy: 0.6543
Epoch 9/50
24/24 [==============================] - 0s 653us/step - loss: 0.5799 - accuracy: 0.7141
Epoch 10/50
24/24 [==============================] - 0s 653us/step - loss: 0.6144 - accuracy: 0.6981
Epoch 11/50
24/24 [==============================] - 0s 653us/step - loss: 0.5483 - accuracy: 0.7340
Epoch 12/50
24/24 [==============================] - 0s 653us/step - loss: 0.5948 - accuracy: 0.7168
Epoch 13/50
...
Epoch 50/50
24/24 [==============================] - 0s 697us/step - loss: 0.3916 - accuracy: 0.8384
21/21 [==============================] - 0s 651us/step - loss: 0.4499 - accuracy: 0.8093
Loss: 0.4498700797557831, Accuracy: 0.8093023300170898
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Neural Network AUC-PR/Loss



Neural Network





Model Evaluation Metrics (Neural Network)

Metric	Train	Test
Accuracy	0.873670212765957	0.817054263565892
AUC-ROC	0.943143301007438	0.885153918482482
AUC-PR	0.894623118192099	0.823457511350256
F1 Score	0.802494802494802	0.734234234234234
F0.5 Score	0.839130434782609	0.780651340996168
F2 Score	0.768924302788844	0.693027210884354

```
Maximum F2 Scores optimized for recall (Neural Network):
Train Set: Max F2 = 0.8639 at threshold 0.20
Test Set: Max F2 = 0.8425 at threshold 0.17
```

Conclusion

Final Model Comparison Table (Results analysis)

Model	Train F2	Threshold	Test F2	Threshold	Overfitting (gap)
Decision Tree	0.9666	0.02	0.9220	0.34	High
Gradient Boost	0.9969	0.45	0.9626	0.31	Low
Random Forest	0.9889	0.36	0.9437	0.30	Moderate
Neural Network	0.8639	0.20	0.8425	0.17	Very low/No

The Best Overall

Gradient Boosting

Highest F2 score: 0.962
Best Recall performance,
Good generalization and stable

Special mention

Neural Network

Lowest F2 Score: 0.8425
Weakest Recall, but **highest generalisation and most stability**

Average

Random Forest

2nd Place F2 Score: 0.944
OK Recall performance,
Moderate overfitting, Stable

The Worst

Single Decision Tree

3rd Place F2 Score: 0.922
Weak Recall performance,
Highest overfitting, Unstable

Questions?

Work distribution:

Hudson - Data cleaning + Data analysis + some of metrics + Conclusion

Jia Wei - Metrics + Decision tree + Gradient boost

Anu - Random Forest

Randall - Neural Network/MLP

Everyone - Debugging code

References

Canales, J. J., & Halliday, G. (2022). *Sounding the alarm on a future epidemic: Alzheimer's disease. Frontiers in Aging Neuroscience*, *14*, 937486. https://doi.org/10.3389/fnagi.2022.937486

UCLA Newsroom. (n.d.). *Sounding the alarm on a future epidemic: Alzheimer's disease*. UCLA Newsroom. https://newsroom.ucla.edu/stories/sounding-the-alarm-on-a-future-epidemic:-alzheimer-s-disease

Tumbleweed. (2017, September 14). *What is a threshold in a precision-recall curve?* Stack Overflow. https://stackoverflow.com/questions/46224752/what-is-a-threshold-in-a-precision-recall-curve

scikit-learn developers. (n.d.). *Plot precision-recall curve. scikit-learn Documentation*. https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html

Gaurav, P. (2021, March 16). *Mastering classification metrics: A beginner's guide – Part 2 (F1, F0.5, and F2 scores)*. Medium. https://medium.com/@prateekgaurav/mastering-classification-metrics-a-beginners-guide-part-2-f1-f0-5-and-f2-scores-154d5c729085

Brownlee, J. (2020, August 14). *Tour of evaluation metrics for imbalanced classification. Machine Learning Mastery*. https://machinelearningmastery.com/tour-of-evaluation-metrics-for-imbalanced-classification/

Brownlee, J. (2020, August 14). *ROC curves and precision-recall curves for imbalanced classification. Machine Learning Mastery*. https://machinelearningmastery.com/roc-curves-and-precision-recall-curves-for-imbalanced-classification/

user3780539. (2017, November 20). *Explanation of the F-beta formula. Data Science Stack Exchange*. https://datascience.stackexchange.com/questions/22234/explanation-of-the-f-beta-formula

XGBoost developers. (n.d.). *XGBoost tutorials – Model. XGBoost Documentation*. https://xgboost.readthedocs.io/en/latest/tutorials/model.html

El Kharoua, R. (2022). *Alzheimer's Disease Dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/rabieelkharoua/alzheimers-disease-dataset

References

Optuna. (n.d.). *Optuna: A hyperparameter optimization framework*. https://optuna.org/#code_examples

Optuna. (n.d.). *Optuna tutorial*. https://optuna.readthedocs.io/en/stable/tutorial/

Corochann. (n.d.). *Optuna tutorial for hyperparameter optimization* [Kaggle Notebook]. Kaggle. https://www.kaggle.com/code/corochann/optuna-tutorial-for-hyperparameter-optimization

scikit-learn developers. (n.d.). *User guide*. scikit-learn. https://scikit-learn.org/stable/user_guide.html

TensorFlow. (n.d.). *TensorFlow guide*. https://www.tensorflow.org/guide

TensorFlow. (n.d.). *TensorFlow tutorials*. https://www.tensorflow.org/tutorials

TutorialsPoint. (n.d.). *TensorFlow tutorial*. https://www.tutorialspoint.com/tensorflow/index.htm

DataCamp. (n.d.). *TensorFlow tutorial*. https://www.datacamp.com/tutorial/tensorflow-tutorial

Tubelwj. (2018, April 11). *Scikit-plot: Making machine learning model visualization easier*. Medium. https://medium.com/@tubelwj/scikit-plot-making-machine-learning-model-visualization-easier-f53133c7fd51